



ULPGC

**Universidad de
Las Palmas de
Gran Canaria**



eii

**ESCUELA DE
INGENIERÍA INFORMÁTICA**

Visión Por Computador

Trabajo Final: Plataformero 2D

Autores

Leslie Liu Romero Martín

María Cabrera Végez

Índice general

1. Introducción	1
1.1. Motivación del trabajo	1
1.2. Objetivo de la propuesta	1
1.3. Organización del documento	2
2. Descripción técnica del trabajo	3
2.1. Arquitectura del sistema	3
2.2. Módulo de procesamiento de visión	3
2.2.1. Adquisición y preprocesamiento	3
2.2.2. Detección de puntos clave (<i>landmarks</i>)	3
2.2.3. Lógica de reconocimiento de gestos	4
2.3. Interfaz de comunicación	5
2.3.1. Serialización	5
2.3.2. Configuración de red	5
2.3.3. Frecuencia de envío	5
2.4. Integración en el motor de videojuegos	5
2.4.1. Recepción de mensajes	5
2.4.2. Mapeo de comandos	6
3. Fuentes y tecnologías utilizadas	7
3.1. MediaPipe	7
3.2. OpenCV (Python)	7
3.3. Godot	7
4. Conclusiones y propuestas de ampliación	9
5. Herramientas con las que nos hubiera gustado contar	10
6. Diario de reuniones	11
6.1. 22 de noviembre de 2025	11
6.2. 14 de diciembre de 2025	11
6.3. 21 de diciembre de 2025	11
6.4. 27 de diciembre de 2025	11
6.5. 28 de diciembre de 2025	12
6.6. 2 de enero de 2026	12
6.7. 4 de enero de 2026	12
6.8. 9 de enero de 2026	12

Índice de figuras

2.1. Captura de movimiento del usuario y transmisión de comandos al video- juego en tiempo real.	3
---	---

Índice de tablas

2.1. Mapeo de comandos	6
----------------------------------	---

Capítulo 1

Introducción

En el presente trabajo se ha desarrollado una propuesta para el uso de la Visión por Computador como controlador de un videojuego, más concretamente, un "plataformero 2D". Los controles se centran en el uso de los brazos y el movimiento del cuerpo para desencadenar el movimiento del personaje en la escena. Se captura el movimiento a través de una "webcam" se reflejan los movimientos acorde a la detección de la pose de la persona con un modelo pre-entrenado de MediaPipe.

1.1. Motivación del trabajo

La idea de este trabajo representa una combinación entre pasión - videojuegos - y la búsqueda de un proyecto que aportase más que una experiencia de aprendizaje. Tras haber podido apreciar la Visión por Computador como una posible fuente de entretenimiento al aplicarla en casos de uso no tan técnicos como puede ser el análisis de datos, sino en aspectos más cercanos, como un juego de trivias o un filtro de vídeo, se abre un abanico de posibilidades.

Tras esta reflexión, surge la idea de un videojuego en el que se puedan reflejar los movimientos de una persona delante de una cámara en movimientos de un personaje, esto convierte un juego que en un inicio puede ser simple en una experiencia más inmersiva, entretenida e innovadora.

Un plataformero en 2D fue la conclusión más coherente, para enfocar la atención en el uso de la visión por computador y su papel en este proyecto, en vez de en el diseño y la gran complejidad que supone crear un videojuego desde cero, manteniendo el aspecto visual del juego que le da vida a la idea sin perder de vista el objetivo de esta tarea.

1.2. Objetivo de la propuesta

Esta propuesta tiene como objetivo mostrar uno de los aspectos de la visión por computador que no es tan común o visible hoy en día, utilizar elementos audiovisuales para cambiar la forma en la que interactúan las personas con las máquinas. Generalmente existen múltiples periféricos para poder comunicar órdenes a los ordenadores, como puede ser un ratón o un teclado, sin embargo, la cámara y las imágenes suelen cumplir un papel distinto, el de transmitir mensajes entre personas, siendo el computador un intermediario. Este proyecto demuestra que, con las herramientas adecuadas, la entrada de vídeo e imágenes pueden servir como fuente de información para los ordenadores. Más

concretamente en el caso de los videojuegos, esta nueva perspectiva sustituye a un mando o controlador específico, que puede representar una complicación para usuarios que no tienen experiencia previa, e incluso restringir la accesibilidad a usuarios con limitaciones físicas.

1.3. Organización del documento

La estructura de esta memoria se divide en *ocho* capítulos, incluyendo el actual, que explican: la motivación y objetivo de este trabajo, la descripción técnica en profundidad del proyecto, fuentes y tecnologías utilizadas, conclusiones que se han obtenido tras la realización de dicho proyecto así como propuestas de ampliación, herramientas utilizadas junto con aquellas que se habrían utilizado si hubiese sido posible y por último un diario de las reuniones periódicas del grupo.

Capítulo 2

Descripción técnica del trabajo

2.1. Arquitectura del sistema

El sistema se ha dividido en dos módulos: el módulo de visión y el módulo de juego.

En el módulo de visión se ha usado Python, con la ayuda de las bibliotecas de OpenCV y MediaPipe. Su función principal es la captura de movimiento en tiempo real a través de una cámara. Cada fotograma se procesa para seguir las partes del cuerpo que interesan para la formación de una pose concreta. Se obtiene la postura realizada por el usuario y se envía al juego.

En el módulo de juego se ha usado Godot como motor, encargándose este de la gestión de las escenas, la representación gráfica y la interfaz del usuario. En pocas palabras, recibe los datos generados por el sistema de visión y los traduce en acciones dentro del juego, moviendo al personaje a lo largo del mapa.

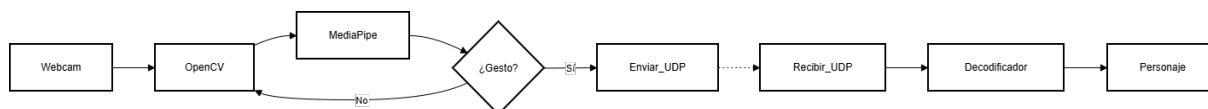


Figura 2.1: Captura de movimiento del usuario y transmisión de comandos al videojuego en tiempo real.

2.2. Módulo de procesamiento de visión

2.2.1. Adquisición y preprocesamiento

Para la captura de la imagen, se utiliza la webcam disponible del ordenador en el que se ejecute el script de Python, la resolución depende de la calidad de cada cámara. En cuanto al preprocesado, se convierte cada frame procesado por OpenCV de BGR a RGB al pasárselo al modelo de MediaPipe. La imagen se le muestra al usuario tal y como se obtiene desde la cámara, se ha evitado mostrar el resultado en modo espejo ya que esto podía ocasionar errores de cálculo, para remediarlo, se indica con anotaciones cuál es el brazo derecho y cuál es el izquierdo.

2.2.2. Detección de puntos clave (*landmarks*)

La detección de puntos clave o *landmarks* recae sobre MediaPipe Pose, un modelo pre-entrenado que se encarga de detectar hasta 33 puntos de referencia del cuerpo humano.

El proceso de detección se divide en dos fases, la detección de la región de interés (ROI, por sus siglas en inglés) y la detección de dichos puntos clave. Para este proyecto en concreto, solo se han utilizado 12 de los 33, ambos hombros (derecho e izquierdo), ambos codos, ambas muñecas, los dos lados de la cadera, las rodillas y los tobillos. Estos puntos clave han aportado suficiente información para que la detección sea fiable en la gran mayoría de los casos, con excepción de situaciones de iluminación desfavorables o posiciones delante de la cámara que no permitan la visibilidad completa de las extremidades mencionadas.

2.2.3. Lógica de reconocimiento de gestos

Por último en el módulo de procesamiento de visión, queda la lógica de reconocimiento de los gestos. Se trata de la sección más crucial en este módulo para el correcto funcionamiento del proyecto. Para ello, se han creado funciones correspondientes para cada uno de los cálculos, lo que ha facilitado su uso. Existen cuatro funciones para la detección de gestos y una función auxiliar para el cálculo de los ángulos.

Funciones principales

- *arm_raised(shoulder, elbow, wrist)*: Detecta si un brazo (derecho o izquierdo), se encuentra levantado verticalmente. Toma como parámetros los puntos de interés del hombro, el codo y la muñeca y calcula el ángulo que forman los tres puntos para comprobar que este sea superior a 160° (dejando cierto margen con respecto a la horizontal perfecta). Asimismo, como cálculo adicional, se comprueba la diferencia entre las coordenadas X de los tres puntos y se comprueba su alineación con un margen de 0.05. Por último, se comprueba que la posición de la muñeca se encuentra por encima de la del hombro para asegurar que el brazo se encuentra hacia arriba a través de la coordenada Y. Si las tres condiciones se dan simultáneamente, se considera que el brazo se encuentra levantado.
- *arm_horizontal(shoulder, elbow, wrist)*: Detecta si un brazo se encuentra levantado horizontalmente. Sus parámetros son también el hombro, el codo y la muñeca. Se calcula nuevamente el ángulo para que sea superior a 160° , sin embargo, para calcular la horizontalidad, se utiliza la coordenada Y. Para esta condición, se tienen en cuenta solamente estas dos premisas, si ambas se cumplen, se considera que el brazo se encuentra horizontal.
- *crouched(hip, knee, ankle)*: Detecta si el usuario se encuentra agachado. Para ello, utiliza la posición de la cadera, la rodilla y el tobillo. Se calcula el ángulo entre estos tres puntos y la distancia vertical entre la cadera y el tobillo (esta distancia tiende a reducirse cuando la persona se agacha). Si el ángulo es menor a 100° o la distancia es menor a 0.3, se considera que la persona se está agachando.
- *is_jumping*: Detecta si el usuario se encuentra realizando los gestos necesarios para ejecutar un salto del personaje. Un salto hacia la derecha se realiza si el usuario levanta el brazo izquierdo en vertical y el derecho en horizontal, por el contrario, un salto hacia la izquierda se realiza si el usuario levanta el brazo derecho en vertical y el izquierdo en horizontal. En esta función, se utilizan las funciones ya existentes *arm_raised()* y *arm_horizontal()* para comprobar las acciones del usuario.

Funciones auxiliares

- *calculate_angle(a, b, c)*: Calcula el ángulo entre la recta AB y la recta BC. Para ello se utiliza la diferencia de las tangentes inversas de los vectores formados, tal y como se describe en la Ecuación 2.1.

$$\theta = \left| (\arctan2(C_y - B_y, C_x - B_x) - \arctan2(A_y - B_y, A_x - B_x)) \cdot \frac{180}{\pi} \right| \quad (2.1)$$

Donde (x, y) representan las coordenadas normalizadas de cada punto clave. Posteriormente, si el ángulo resultante excede los 180° , se calcula su suplementario para asegurar que siempre se trabaja con el ángulo interior.

2.3. Interfaz de comunicación

2.3.1. Serialización

Para enviar los datos, estos se empaquetan en un diccionario cuyas claves representan una acción del personaje (p.e. ‘move_right’) y cuyos valores son enteros que pueden ser binarios o de múltiple opción. Estos datos se convierten de un diccionario de Python a formato JSON utilizando el módulo ‘json’ nativo de Python para poder ser enviados.

2.3.2. Configuración de red

La comunicación entre Visual Studio y Godot se realiza mediante comunicación por red usando el protocolo UDP. Permite un intercambio de información eficiente y rápido, pero para ello es necesario hacer una serie de configuraciones. En primer lugar, se debe realizar el puerto por el que Godot escucha. Para ello, se debe ir al apartado “Proyecto” y a “Configuración del Proyecto”. Una vez dentro, se busca el apartado de network (debe de estar activo la configuración avanzada). Dentro del apartado debug se revisa el “Remote Port”. En el caso del proyecto, se trata del puerto 6005.

Python envía los datos al mismo puerto mediante un socket UDP, permitiendo que Godot pueda recibirlos y que los vaya procesando en tiempo real.

2.3.3. Frecuencia de envío

Los mensajes se envían cada vez que se detecta una pose predefinida, lo que hace que el juego sea interactivo y tenga un tiempo de respuesta muy bajo.

Cada vez que sea detectada una pose válida mediante MediaPipe, se enviará un mensaje a Godot. Este indicará la acción realizada y puede garantizar cierta rapidez en el reflejo de las acciones del jugador al personaje

2.4. Integración en el motor de videojuegos

2.4.1. Recepción de mensajes

Godot utiliza un servidor UDP que se encuentra escuchando al puerto 6005. En cada frame, el servidor debe de comprobar si se le han enviado datos. En caso de tener, los recibe y los convierte a datos que pueda usar. Trata la información para poder obtener los comandos enviados y poder indicar al personaje cómo debe moverse.

2.4.2. Mapeo de comandos

Se recibe un diccionario donde cada clave corresponde a una acción que el personaje debe realizar (move_right, attack, slide, jump, move_left). Según el valor que se tenga cada clave, Godot indicará al script del Player que debe hacer una acción u otra (Tabla 2.1).

Tabla 2.1: Mapeo de comandos

Comando recibido	Valor que lo activa	Función en Godot	Acción que realiza
move_right	0	_move_x("derecha")	Movimiento hacia la derecha
move_left	1	_move_x("izquierda")	Movimiento hacia la izquierda
jump	1	_jump(1)	Salto a la derecha
jump	2	_jump(2)	Salto a la izquierda
slide	1	_slide()	Agacharse
attack	1	_attack()	Atacar

Capítulo 3

Fuentes y tecnologías utilizadas

A pesar de que en el planteamiento inicial pensamos en utilizar YOLO Pose como nuestro detector de pose, tras varias pruebas comparativas nos decidimos por usar MediaPipe, por lo que las tecnologías utilizadas serían: MediaPipe para la detección y seguimiento de la pose en tiempo real, OpenCV (Python) para el procesamiento lógico de los datos que aporta el modelo de detección de MediaPipe y Godot como el motor de videojuegos para crear nuestro plataformero

3.1. MediaPipe

MediaPipe se basa en una plataforma de código abierto desarrollado por Google que permite evaluar, integrar y personalizar modelos de visión artificial en una aplicación. Se especializa en el seguimiento y detección del cuerpo humano en tiempo real.

3.2. OpenCV (Python)

Para el desarrollo del proyecto se ha usado Python como el lenguaje principal de programación para poder usar MediaPipe. Esto se debe a su sencillez y legibilidad. Cuenta con una gran cantidad de bibliotecas compatibles, lo que facilita la implementación y el mantenimiento del código.

OpenCV es una biblioteca de código abierto especializada en el procesamiento de imágenes y vídeos en tiempo real. Dentro de Python, se usa a través del módulo `cv2`, que proporciona el uso de la captura de vídeo desde la cámara, el procesamiento de imágenes, la detección de objetos y la visualización de resultados.

Se ha optado por OpenCV debido a su alto rendimiento y compatibilidad multiplataforma. Además, permitía el uso de MediaPipe. La combinación permite capturar imágenes con OpenCV, analizar con modelos avanzados y mostrar los resultados de forma eficiente.

3.3. Godot

Godot se trata de un motor de desarrollo de videojuegos de código abierto que permite crear juegos tanto en 2D como en 3D. Cuenta con un editor visual intuitivo y compatibilidad multiplataforma, permitiendo conectar el proyecto de Godot con el código de Visual Studio.

El motor ha sido empleado en diversos juegos. Algunos son Sonic Colors: Ultimate, Commander Keen in Keen Dreams, etc.

La elección de Godot frente a otras tecnologías se ha debido, principalmente, al hecho de que una de las integrantes del grupo ya lo conocía y lo había usado con anterioridad. De esta manera, no era completamente necesario aprender su funcionamiento desde cero. Por otro lado, Godot es bastante completo, ofrece herramientas integradas como una sistema de físicas, gestión de escenas, animaciones, sistema de señales...

Cabe añadir que Godot usa GDScript, un lenguaje parecido a Python, que es sencillo de aprender, especialmente en desarrolladores con experiencia previa en Python.

Capítulo 4

Conclusiones y propuestas de ampliación

Tras completar este trabajo, se puede concluir que los objetivos han sido cumplidos, el funcionamiento y la dinámica del videojuego es la deseada. Los controles son interactivos, responden de manera eficiente a los gestos realizados por el usuario y permiten que el jugador pueda tener una experiencia de inmersión más lograda, teniendo control total sobre el personaje. Más importante aún, se ha alcanzado la meta de buscar en la Visión por Computador un proyecto que constituye la combinación ideal de aprendizaje y entretenimiento con el objetivo de crear un producto final funcional y viable.

Se ha logrado aplicar conceptos de aprendidos durante el curso, al igual que sobre el desarrollo de videojuegos. Se ha obtenido experiencia en la comunicación en tiempo real de sistemas, en el uso de UDP para interactuar rápidamente y en el diseño de sistemas para separar por partes la lógica de un juego, facilitando futuras ampliaciones que se quisieran hacer.

Sin embargo, a pesar de ello, el tiempo no ha sido ilimitado y han existido ciertos obstáculos para alcanzar el potencial completo de este proyecto. Los comandos más básicos se han mapeado y el juego es funcional y entretenido, pero tanto la sección de Visión por Computador como la sección del juego podrían extenderse.

Como propuestas de ampliación, se pueden añadir más comandos como el ataque o la defensa, enemigos que combatir, mapas más largos, un sistema de niveles, entre otros.

Se trata de una propuesta flexible donde cabe mucha mejora. Con dedicación a largo plazo, las posibilidades serían infinitas, pues el sistema podría llegar a evolucionar a un videojuego más complejo, con la posibilidad de más gestos, interacciones, etc. En pocas palabras, con una experiencia de usuario más rica en elementos, aprovechando de mejor manera la unión de la visión por computador y el entretenimiento digital.

Capítulo 5

Herramientas con las que nos hubiera gustado contar

Si se hubiese contado con más tiempo, se habría querido usar más tecnologías para dar un aire más profesional al trabajo realizado. Entre estas, se encuentran mejores cámaras con una calidad mayor para una detección superior del movimiento del usuario. El uso de herramientas profesionales de diseño para la UI/UX, como es Figma, pues habría permitido diseñar de forma más cómoda la interfaz, antes de intentar directamente implementarla. Por otro lado, habría sido interesante poder añadir audio, como algo de música, para acompañar al jugador y que este no se quede en silencio moviéndose.

Capítulo 6

Diario de reuniones

Se han realizado reuniones periódicas para la realización completa del trabajo, generalmente una o dos veces a la semana, con una duración media de 4 horas, de manera telemática. Cada reunión se ha enfocado en un aspecto del trabajo, desde el planteamiento inicial y la presentación de la propuesta con el ante-proyecto hasta el producto final. Se ha seguido un proceso incremental, dedicando tiempo a la realización de pruebas y experimentación con las herramientas para familiarizarse con el entorno de trabajo.

6.1. 22 de noviembre de 2025

Desarrollo del ante-proyecto, planteamiento de la idea y tecnologías a utilizar. Creación de la presentación para la posterior exposición en clase.

6.2. 14 de diciembre de 2025

Realización de pruebas con las tecnologías propuestas, comparativa entre YOLO y MediaPipe, nos decidimos por utilizar MediaPipe. Primeros pasos con Godot (creación del proyecto, script base)

6.3. 21 de diciembre de 2025

Pruebas de compatibilidad y conectividad entre Godot y nuestro script de Python para poder establecer una comunicación lo suficientemente rápida para que el proyecto sea viable. Finalmente se opta por el uso de sockets (puertos).

6.4. 27 de diciembre de 2025

Implementación del script de Python: carga del modelo de MediaPipe, lectura de la postura, pruebas para comprobar la fiabilidad de los resultados con respecto a las necesidades de nuestro proyecto. Los resultados son favorables, se puede claramente leer la postura incluso en condiciones de iluminación no ideales y con cámaras de calidad media. Se establece la conexión con Godot a través de puertos y todo funciona correctamente. En Godot, avanzamos con el proyecto, ya existe una escena base.

6.5. 28 de diciembre de 2025

Completamos el script de Python, se finalizan todas las funciones para detectar las posturas que se establecieron inicialmente (detección de brazos en posición horizontal y vertical, agachado, combinación de varias posturas) que posteriormente se conectan a una acción del personaje en Godot. Al final de esta sesión el personaje responde a los movimientos de la persona utilizando la webcam de manera prácticamente inmediata, ya existe una escena por la que moverse y un personaje capaz de desplazarse y saltar.

6.6. 2 de enero de 2026

Se añaden las funcionalidades faltantes como la recolección de monedas y se comienza a documentar el proyecto. Para las monedas, primero se hizo la animación de las monedas para que se vean dando vueltas sobre un mismo punto. A continuación, se pudo modificar el código para poder recoger las monedas, contando con un contador que va siguiendo al usuario.

6.7. 4 de enero de 2026

Reunión presencial, se prueban las funcionalidades implementadas hasta el momento, se graban vídeos al aire libre para poder acumular contenido para el vídeo de la entrega y la presentación del trabajo. Tras muchas pruebas, se añaden funcionalidades como los *checkpoints* y los pinchos como enemigos; ambos se colocan de manera estratégica para que el juego se pueda superar.

6.8. 9 de enero de 2026

Dedicación completa a terminar la documentación del trabajo. Se continúa donde se dejó algunas reuniones atrás, se crea la memoria con LaTeX, se editan los vídeos tomados en la reunión anterior y se realiza la presentación para la defensa del trabajo.